Repeat CIA 1 LAB Component

1. Load, inspect, and verify data types

In [27]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from IPython.display import display

sns.set_theme(style="whitegrid")
pd.set_option("display.max_columns", None)
raw = pd.read_csv("housing.csv")
df = pd.DataFrame({
    "MedInc": raw.median_income,
    "HouseAge": raw.housing_median_age,
    "AveRooms": raw.total_rooms / raw.households,
    "AveBedrms": raw.total_bedrooms / raw.households,
    "Population": raw.population,
    "AveOccup": raw.population / raw.households,
    "Latitude": raw.latitude,
    "Longitude": raw.longitude,
    "MedHouseVal": raw.median_house_value / 100000,
})
assert len(df) == len(raw) and df.shape[1] == 9
```

In [28]:
```
display(df.head(10))
print("Continuous numerical features:", df.columns.drop("MedHouseVal").tolist())
print("Target: MedHouseVal (median house value in $100,000 units)")
display(df.dtypes.rename("dtype").to_frame())
print("All analysis columns are numeric; ocean_proximity is categorical.")
```

	MedInc	HouseAge	AveRooms	AveBedrms	Population	AveOccup	Latitude	Longitu
0	8.3252	41.0	6.984127	1.023810	322.0	2.555556	37.88	-122
1	8.3014	21.0	6.238137	0.971880	2401.0	2.109842	37.86	-122
2	7.2574	52.0	8.288136	1.073446	496.0	2.802260	37.85	-122
3	5.6431	52.0	5.817352	1.073059	558.0	2.547945	37.85	-122
4	3.8462	52.0	6.281853	1.081081	565.0	2.181467	37.85	-122
5	4.0368	52.0	4.761658	1.103627	413.0	2.139896	37.85	-122
6	3.6591	52.0	4.931907	0.951362	1094.0	2.128405	37.84	-122
7	3.1200	52.0	4.797527	1.061824	1157.0	1.788253	37.84	-122
8	2.0804	42.0	4.294118	1.117647	1206.0	2.026891	37.84	-122
9	3.6912	52.0	4.970588	0.990196	1551.0	2.172269	37.84	-122

```
Continuous numerical features: ['MedInc', 'HouseAge', 'AveRooms', 'AveBedrms', 'Popu
lation', 'AveOccup', 'Latitude', 'Longitude']
Target: MedHouseVal (median house value in $100,000 units)
```

	dtype
MedInc	float64
HouseAge	float64
AveRooms	float64
AveBedrms	float64
Population	float64
AveOccup	float64
Latitude	float64
Longitude	float64
MedHouseVal	float64

```
All analysis columns are numeric; ocean_proximity is categorical.
```

2. Descriptive statistics, variance, and skewness

In [29]:
```
display(df.describe().T)
variances = df.var().sort_values(ascending=False)
display(variances.rename("sample_variance").to_frame())
print(f"Largest variance: {variances.index[0]} ({variances.iloc[0]:,.2f}); variance
comparison = df[["MedHouseVal", "MedInc"]].agg(["mean", "median"]).T
display(comparison)
print("Both means exceed their medians, suggesting positive (right) skew.")
```

```
fig, axes = plt.subplots(1, 2, figsize=(12, 4))
for ax, column in zip(axes, ["MedHouseVal", "MedInc"]):
    sns.histplot(df[column], bins=40, kde=True, ax=ax)
    ax.axvline(df[column].mean(), color="red", linestyle="--", label="Mean")
    ax.axvline(df[column].median(), color="green", linestyle=":", label="Median")
    ax.set_title(f"{column} distribution")
    ax.legend()
plt.tight_layout(); plt.show()
```

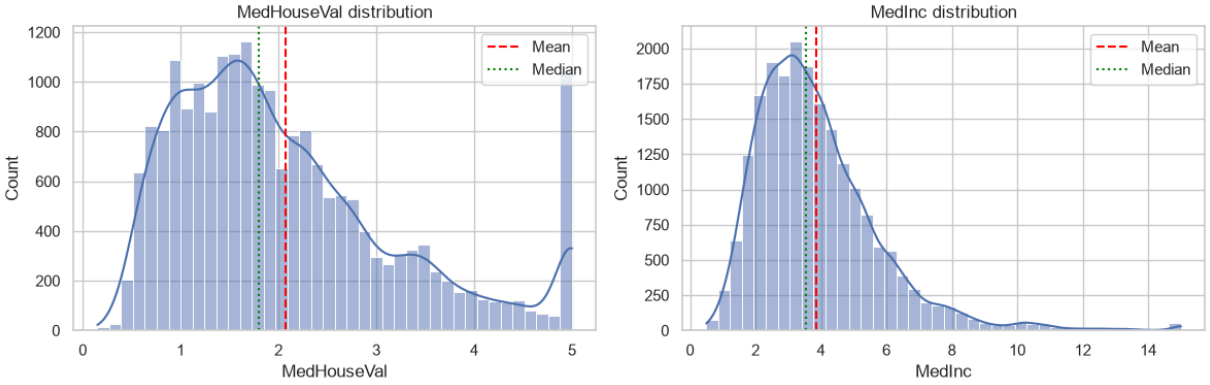
	count	mean	std	min	25%	50%
MedInc	20640.0	3.870671	1.899822	0.499900	2.563400	3.534800
HouseAge	20640.0	28.639486	12.585558	1.000000	18.000000	29.000000
AveRooms	20640.0	5.429000	2.474173	0.846154	4.440716	5.229129
AveBedrms	20433.0	1.097062	0.476104	0.333333	1.006029	1.048889
Population	20640.0	1425.476744	1132.462122	3.000000	787.000000	1166.000000
AveOccup	20640.0	3.070655	10.386050	0.692308	2.429741	2.818116
Latitude	20640.0	35.631861	2.135952	32.540000	33.930000	34.260000
Longitude	20640.0	-119.569704	2.003532	-124.350000	-121.800000	-118.490000
MedHouseVal	20640.0	2.068558	1.153956	0.149990	1.196000	1.797000

	sample_variance
Population	1.282470e+06
HouseAge	1.583963e+02
AveOccup	1.078700e+02
AveRooms	6.121533e+00
Latitude	4.562293e+00
Longitude	4.014139e+00
MedInc	3.609323e+00
MedHouseVal	1.331615e+00
AveBedrms	2.266751e-01

```
Largest variance: Population (1,282,470.46); variance is scale-dependent.
```

	mean	median
MedHouseVal	2.068558	1.7970
MedInc	3.870671	3.5348

```
Both means exceed their medians, suggesting positive (right) skew.
```



3. Missing values and missingness mechanisms

In [30]:
```
missing = pd.DataFrame({"missing_count": df.isna().sum(), "missing_percent": df.isn
total_missing = missing.missing_count.sum()
display(missing)
print(f"Total missing cells: {total_missing} ({total_missing / df.size * 100:.3f}%
```

	missing_count	missing_percent
MedInc	0	0.000000
HouseAge	0	0.000000
AveRooms	0	0.000000
AveBedrms	207	1.002907
Population	0	0.000000
AveOccup	0	0.000000
Latitude	0	0.000000
Longitude	0	0.000000
MedHouseVal	0	0.000000

```
Total missing cells: 207 (0.111% of all cells)
```

AveRooms contained no missing values. AveBedrms contained 207 missing values which was about 1% of the dataset. Since the missing values are below 5%, I would keep the column and impute the missing values with median.

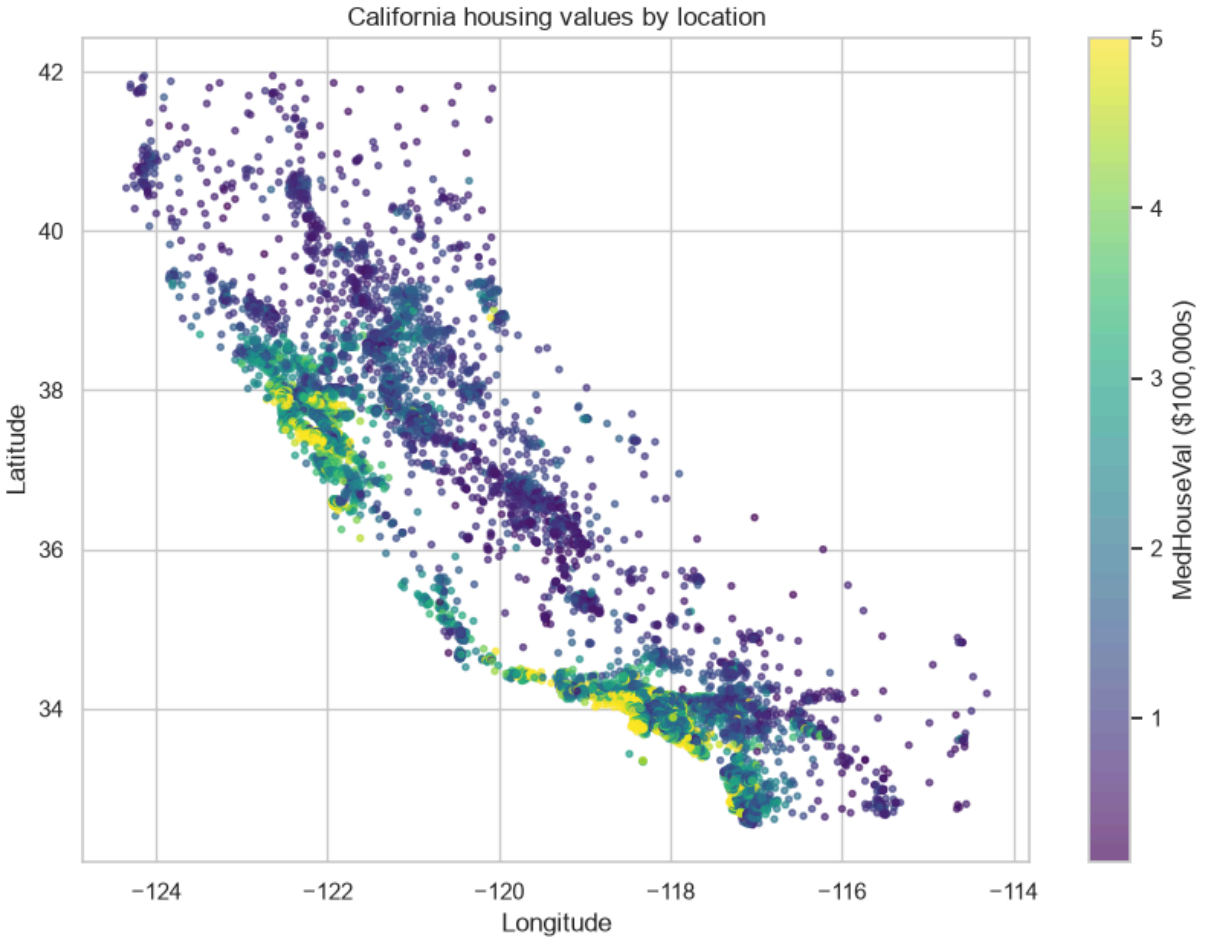
MAR Example : Price evaluations may be missing more often for inland homes, while location is observed. MNAR example: Owners of unusually expensive homes may withhold their valuations, so missingness depends on the unreported price itself. MNAR needs sensitivity analysis because ordinary imputation can remain biased.

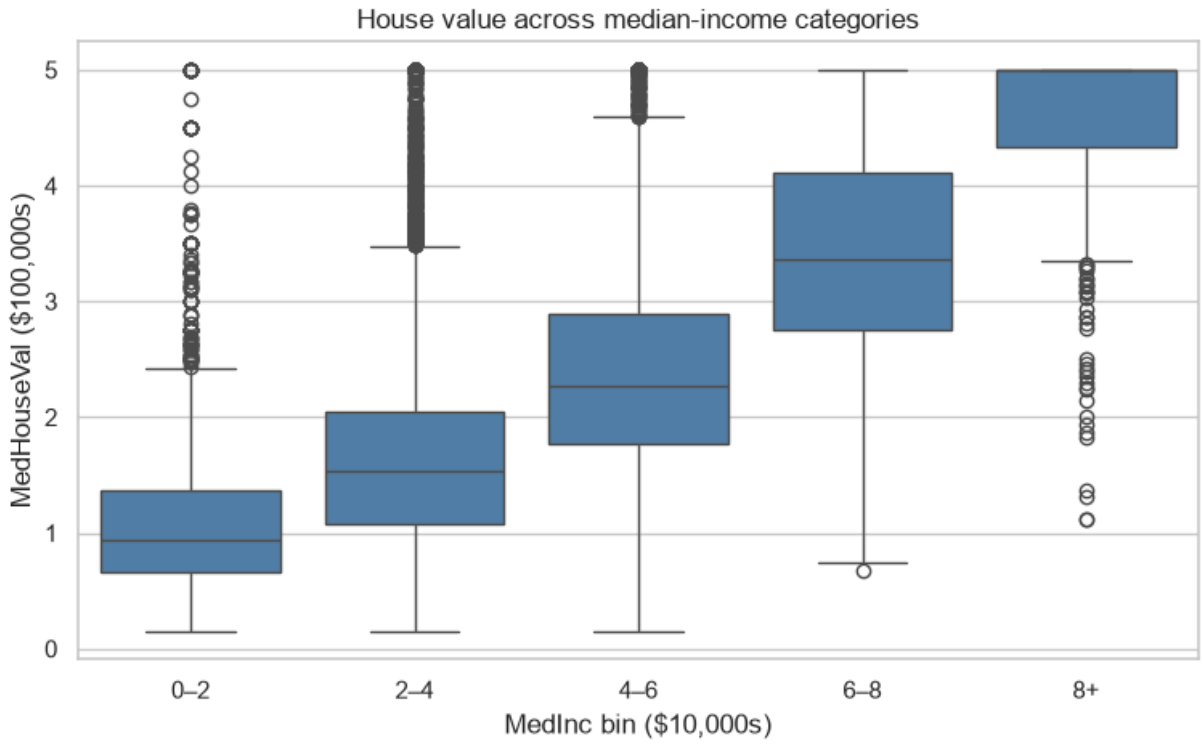
4. Spatial, income-bin, and scatter plots

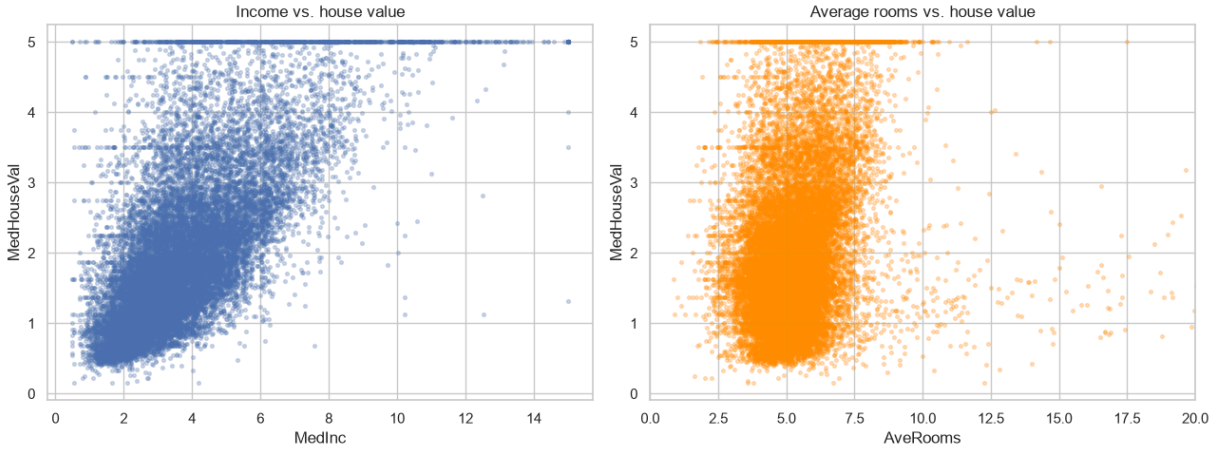
In [31]:
```
plt.figure(figsize=(10, 7))
points = plt.scatter(df.Longitude, df.Latitude, c=df.MedHouseVal, cmap="viridis", s
plt.colorbar(points, label="MedHouseVal ($100,000s)")
plt.title("California housing values by location")
plt.xlabel("Longitude"); plt.ylabel("Latitude"); plt.show()

income_bin = pd.cut(df.MedInc, [0, 2, 4, 6, 8, np.inf], labels=["0–2", "2–4", "4–6"
plt.figure(figsize=(9, 5))
sns.boxplot(x=income_bin, y=df.MedHouseVal, color="steelblue")
plt.title("House value across median-income categories")
plt.xlabel("MedInc bin ($10,000s)"); plt.ylabel("MedHouseVal ($100,000s)"); plt.sho

fig, axes = plt.subplots(1, 2, figsize=(13, 5))
axes[0].scatter(df.MedInc, df.MedHouseVal, s=7, alpha=.25)
axes[0].set(xlabel="MedInc", ylabel="MedHouseVal", title="Income vs. house value")
axes[1].scatter(df.AveRooms, df.MedHouseVal, s=7, alpha=.25, color="darkorange")
axes[1].set(xlabel="AveRooms", ylabel="MedHouseVal", title="Average rooms vs. house
plt.tight_layout(); plt.show()
```





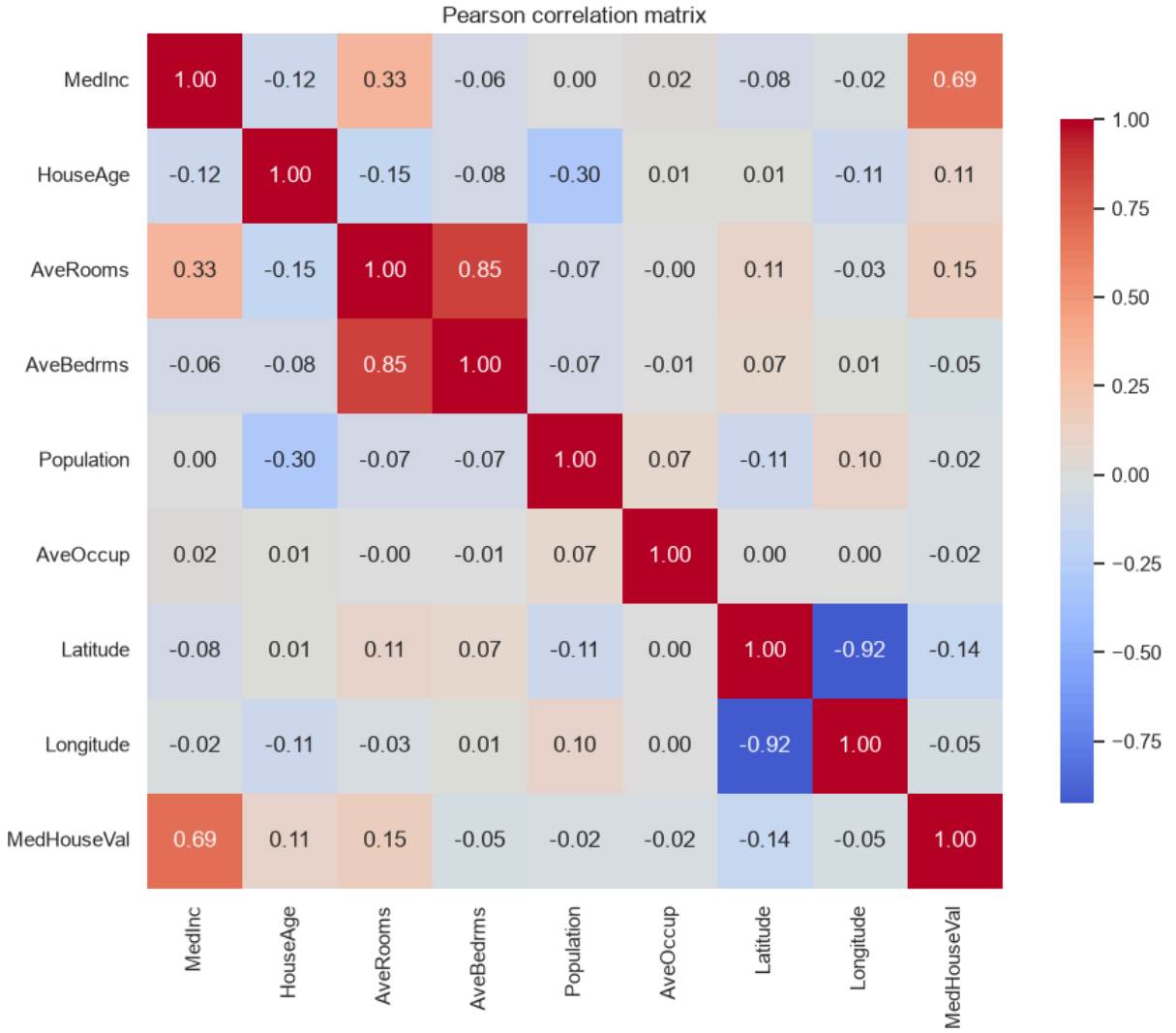


Interpretation: Coastal areas are generally more expensive. House value rises with income, while average rooms has a weaker relationship. The band near 5 is the dataset's price cap.

5. Pearson correlation matrix

In [32]:
```
corr = df.corr()
display(corr.round(3))
plt.figure(figsize=(10, 8))
sns.heatmap(corr, cmap="coolwarm", center=0, annot=True, fmt=".2f", square=True, cb
plt.title("Pearson correlation matrix"); plt.tight_layout(); plt.show()
display(corr.loc[["MedInc", "AveRooms", "HouseAge"], ["MedHouseVal"]].rename(column
```

	MedInc	HouseAge	AveRooms	AveBedrms	Population	AveOccup	Latit
MedInc	1.000	-0.119	0.327	-0.062	0.005	0.019	-0.
HouseAge	-0.119	1.000	-0.153	-0.078	-0.296	0.013	0.
AveRooms	0.327	-0.153	1.000	0.849	-0.072	-0.005	0.
AveBedrms	-0.062	-0.078	0.849	1.000	-0.067	-0.006	0.
Population	0.005	-0.296	-0.072	-0.067	1.000	0.070	-0.
AveOccup	0.019	0.013	-0.005	-0.006	0.070	1.000	0.
Latitude	-0.080	0.011	0.106	0.070	-0.109	0.002	1.
Longitude	-0.015	-0.108	-0.028	0.013	0.100	0.002	-0.
MedHouseVal	0.688	0.106	0.152	-0.047	-0.025	-0.024	-0.



	r with MedHouseVal
MedInc	0.688075
AveRooms	0.151948
HouseAge	0.105623

- MedInc has the strongest positive correlation with price (about 0.69).
- AveRooms (about 0.15) and HouseAge (about 0.11) are weakly positive. -Non encoded categorical features cannot be used in Pearson's corerelation matrix as Correlation is not causation. Pearson correlation requires numeric values with meaningful arithmetic; raw category names cannot be used, and arbitrary integer labels would create a false order.

6. IQR outlier detection

In [33]:
```
outlier_rows = []
for column in ["AveRooms", "Population", "AveOccup", "MedHouseVal"]:
    q1, q3 = df[column].quantile([.25, .75])
    iqr = q3 - q1
    lower, upper = q1 - 1.5 * iqr, q3 + 1.5 * iqr
    count = ((df[column] < lower) | (df[column] > upper)).sum()
    outlier_rows.append([column, q1, q3, iqr, lower, upper, count, count / len(df)
outliers = pd.DataFrame(outlier_rows, columns=["feature", "Q1", "Q3", "IQR", "lower
assert outliers.outlier_percent.between(0, 100).all()
display(outliers.round(3))
```

feature	Q1	Q3	IQR	lower_bound	upper_bound	outlier_count	ou
AveRooms	4.441	6.052	1.612	2.023	8.470	511	
Population	787.000	1725.000	938.000	-620.000	3132.000	1196	
AveOccup	2.430	3.282	0.853	1.151	4.561	711	
MedHouseVal	1.196	2.647	1.451	-0.981	4.824	1071	

IQR flags unusual values, not automatic errors. Extreme AveRooms values may represent genuine luxury or low-density properties, but they can also result from incorrect totals or division by very few households. Extreme MedHouseVal values may be genuine expensive homes

7. Synthesis and recommendations

In [34]:
```
coastal_summary = raw.groupby("ocean_proximity").median_house_value.agg(["count", "
display(coastal_summary.round(0))
print(f"Correlation between AveRooms and AveBedrms: {df.AveRooms.corr(df.AveBedrms)
display(corr.MedHouseVal.drop("MedHouseVal").sort_values(key=abs, ascending=False).
```

ocean_proximity	count	mean	median
ISLAND	5	380440.0	414700.0
NEAR BAY	2290	259212.0	233800.0
NEAR OCEAN	2658	249434.0	229450.0
<1H OCEAN	9136	240084.0	214850.0
INLAND	6551	124805.0	108500.0

```
Correlation between AveRooms and AveBedrms: 0.849
```

	correlation_with_target
MedInc	0.688075
AveRooms	0.151948
Latitude	-0.144160
HouseAge	0.105623
AveBedrms	-0.046739
Longitude	-0.045967
Population	-0.024650
AveOccup	-0.023737

- Coastal groups have higher typical prices than inland groups. The five island rows are too less to make an interpretation.
- AveRooms and AveBedrms are strongly correlated (0.85).
- In linear regression this can inflate coefficient uncertainty and make individual coefficients unstable. Keep one feature or use regularization.
- MedInc is the strongest single linear predictor but location also matters.
- Use training-set median imputation, one-hot encode ocean_proximity, validate by geographic groups, inspect outliers and the capped target, and compare a linear baseline with a tree model.